

INSTITUTO TECNOLÓGICO DE VERACRUZ

Nombre de la Asignatura:

- Administración en Base de Datos

Grupo: **AD26 6J3A**

Nombres del alumno:

- Raúl Alberto Guerrero Aguilar
- Num. Control: 23600266

Nombre de la Actividad:

- Reporte de Práctica 3

Carrera: Ingeniería en Sistemas Computacionales

Fecha de Entrega: 30/Septiembre/2026

1. Introducción

Las bases de datos orientadas a grafos modelan la información mediante **nodos** (entidades), **relaciones** (conexiones dirigidas entre entidades) y **propiedades** (atributos en formato clave-valor). Este enfoque resulta especialmente eficiente cuando el valor de los datos reside en la manera en que se conectan entre sí, como ocurre en redes sociales, motores de recomendación, detección de fraude, topologías de red o sistemas de control de acceso. Neo4j es el motor de grafos nativo de referencia y su lenguaje de consulta declarativo, **Cypher**, describe patrones de grafo con una sintaxis visual cercana al arte ASCII.

Este reporte documenta la práctica de laboratorio en la que se desplegó una instancia de Neo4j, se configuró un flujo de trabajo con control de versiones mediante Git y GitHub, y se ejecutó el ciclo completo de operaciones CRUD con Cypher. Todo el trabajo se realizó en el entorno habitual de desarrollo: una máquina virtual con **Linux Mint** ejecutándose sobre **Oracle VirtualBox**.

Al igual que en configuraciones anteriores, se utilizó **Podman** como motor de contenedores, conservando los principios de seguridad y aislamiento que motivaron esa elección:

- **Arquitectura daemonless:** Podman no depende de un demonio central con privilegios elevados (como `dockerd`). Cada contenedor se lanza como un proceso hijo del usuario que lo ejecuta, supervisado por un monitor ligero (`conmon`). Esto elimina un punto único de falla y una superficie de ataque permanente.
- **Ejecución rootless:** los contenedores corren bajo el UID del usuario normal gracias a los *user namespaces*. El usuario `root` dentro del contenedor se traduce a un UID sin privilegios en el anfitrión, de modo que una eventual fuga del contenedor no otorga privilegios administrativos sobre el sistema.

La práctica se organizó en tres fases: (1) despliegue y automatización de Neo4j con Podman, (2) configuración del control de versiones con Git y (3) implementación de operaciones CRUD en Cypher.

2. Objetivos

Objetivo general

Implementar una base de datos orientada a grafos en un contenedor rootless, versionar el trabajo de laboratorio con Git y manipular datos mediante el ciclo CRUD en Cypher.

Objetivos específicos

- Desplegar Neo4j con Podman utilizando persistencia mediante volúmenes locales.
- Automatizar el arranque del contenedor junto con el sistema operativo mediante servicios de usuario de systemd.
- Configurar Git, clonar el repositorio del taller y trabajar en una rama aislada.
- Resolver la autenticación contra GitHub mediante un Token de Acceso Personal (PAT).
- Ejecutar operaciones `CREATE`, `MATCH`, `SET` y `DETACH DELETE` sobre un nodo de tipo `Usuario`.

3. Entorno de trabajo

La siguiente tabla resume los componentes utilizados durante la práctica.

Componente	Descripción
Hipervisor	Oracle VirtualBox
Sistema operativo invitado	Linux Mint (basado en Ubuntu/Debian)
Motor de contenedores	Podman, sin demonio y sin privilegios (rootless)
Base de datos	Neo4j (imagen oficial de la comunidad)
Interfaz de consulta	Neo4j Browser en <code>http://localhost:7474</code>
Lenguaje de consulta	Cypher
Control de versiones	Git con repositorio remoto en GitHub
Orquestación del arranque	systemd (servicios de usuario) y <code>loginctl</code>

4. Fase 1: Despliegue de Neo4j con Podman y automatización

4.1 Preparación del entorno de persistencia

Un contenedor es efímero por diseño: al eliminarlo, su sistema de archivos interno desaparece junto con los datos que almacenó. Para que la base de datos sobreviva al ciclo de vida del contenedor, se crearon previamente cuatro directorios en el `HOME` del usuario, los cuales se montan posteriormente como volúmenes (*bind mounts*).

```
mkdir -p $HOME/neo4j/{data,logs,import,plugins}
```

El comando aprovecha la expansión de llaves (*brace expansion*) de Bash para crear los cuatro directorios en una sola instrucción. La opción `-p` crea los directorios padre necesarios y evita errores si alguno ya existe. La función de cada directorio es la siguiente:

Directorio local	Ruta en el contenedor	Propósito
<code>~/neo4j/data</code>	<code>/data</code>	Archivos de las bases de datos (persistencia principal).
<code>~/neo4j/logs</code>	<code>/logs</code>	Registros de ejecución y diagnóstico.
<code>~/neo4j/import</code>	<code>/var/lib/neo4j/import</code>	Archivos externos (CSV, JSON) para importación con <code>LOAD CSV</code> .
<code>~/neo4j/plugins</code>	<code>/plugins</code>	Extensiones como APOC o Graph Data Science.

4.2 Despliegue del contenedor

Con los directorios listos, se levantó el contenedor exponiendo los puertos **7474** (HTTP, interfaz web Neo4j Browser) y **7687** (Bolt, protocolo binario de conexión para drivers y herramientas), definiendo las credenciales mediante variables de entorno y montando los volúmenes.

```
podman run -d \
  --name neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/CONTRASENA_SEGURA \
  -v $HOME/neo4j/data:/data:Z \
  -v $HOME/neo4j/logs:/logs:Z \
  -v $HOME/neo4j/import:/var/lib/neo4j/import:Z \
  -v $HOME/neo4j/plugins:/plugins:Z \
  docker.io/library/neo4j:latest
```

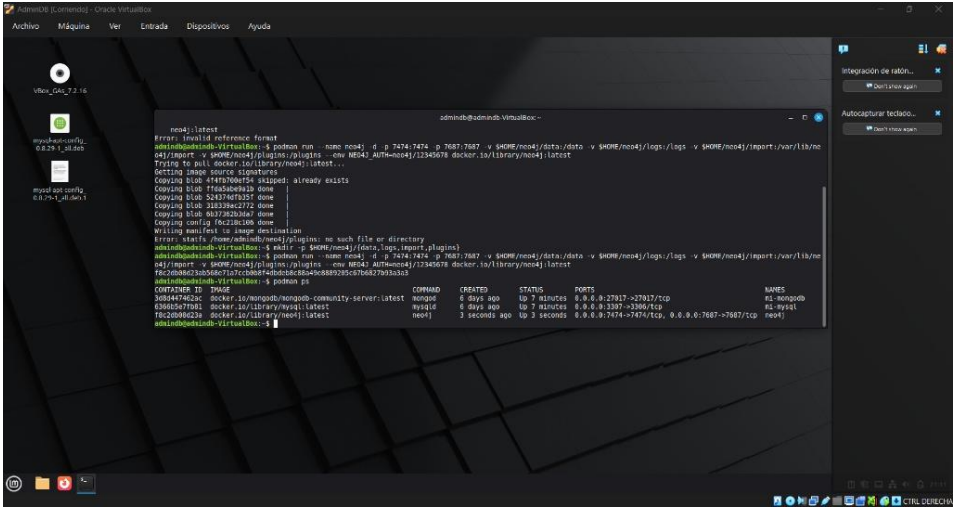
Parámetro	Función
<code>-d</code>	Ejecuta el contenedor en segundo plano (<i>detached</i>).
<code>--name neo4j</code>	Asigna un nombre fijo, necesario para administrarlo y para generar el servicio systemd.
<code>-p 7474:7474, -p 7687:7687</code>	Publica los puertos HTTP y Bolt. Al ser puertos superiores al 1024, no requieren privilegios en modo rootless.
<code>-e NEO4J_AUTH=usuario/clave</code>	Define las credenciales iniciales. Neo4j exige una contraseña de al menos 8 caracteres.
<code>-v origen:destino:Z</code>	Monta cada directorio local como volumen. La opción <code>:Z</code> etiqueta el contenido para SELinux; en sistemas sin SELinux, como Linux Mint, es inocua.
<code>docker.io/library/neo4j:latest</code>	Nombre completo de la imagen, lo que evita la ambigüedad entre registros. Para entornos reproducibles se recomienda fijar una versión específica.

Nota: En el comando, `CONTRASENA_SEGURA` es un marcador de posición que debe sustituirse por la contraseña real. Por seguridad, esta no debe incluirse en repositorios ni capturas de pantalla.

La verificación del despliegue se realizó consultando el estado del contenedor y su bitácora:

```
podman ps
podman logs -f neo4j
```

Cuando la bitácora indica que el servidor ha iniciado y que las interfaces Bolt y HTTP están disponibles, la instancia está lista para recibir conexiones desde el navegador en `http://localhost:7474`.



4.3 Problemas técnicos encontrados y su resolución

Durante el despliegue se presentaron dos inconvenientes comunes, ambos resueltos satisfactoriamente:

Problema	Causa	Solución aplicada
Conflictos por saltos de línea al pegar el comando	Al copiar comandos multilínea, el terminal interpreta cada salto como una pulsación de Enter. Si tras la barra invertida \ queda un espacio o el portapapeles altera el fin de línea, el comando se fragmenta y Bash ejecuta porciones incompletas.	Pegar el comando en una sola línea, o verificar que \ sea el último carácter de cada línea. Como alternativa, guardar el comando en un script con un editor de texto.
Error por falta de directorios previos	Podman no crea automáticamente las rutas de origen de un <i>bind mount</i> . Si <code>~/neo4j/data</code> o los demás directorios no existen, el arranque aborta indicando que la ruta no se encuentra.	Crear la estructura con <code>mkdir -p</code> antes de <code>podman run</code> . Si el intento fallido dejó un contenedor en estado <i>Created</i> , eliminarlo con <code>podman rm neo4j</code> y repetir el comando.

4.4 Automatización del arranque con systemd

Los contenedores rootless se ejecutan dentro de la sesión del usuario; por ello, sin configuración adicional, se detienen al cerrar sesión y no inician al encender el equipo. Para que Neo4j arranque junto con el sistema operativo **sin requerir inicio de sesión**, se combinaron dos mecanismos: el *lingering* de `loginctl` y los servicios de usuario de `systemd`.

Paso 1. Habilitar el lingering del usuario

```
loginctl enable-linger $USER
loginctl show-user $USER | grep Linger
```

`enable-linger` indica a `systemd` que mantenga activo el administrador de servicios del usuario desde el arranque del sistema y aun sin sesiones abiertas. La segunda instrucción confirma el resultado con `Linger=yes`.

Paso 2. Generar la unidad de servicio

```
podman stop neo4j && podman rm neo4j

mkdir -p ~/.config/systemd/user
cd ~/.config/systemd/user
podman generate systemd --name neo4j --files --new
```

Con la opción `--new`, la unidad no depende de un contenedor preexistente: lo crea al iniciar el servicio y lo elimina al detenerlo. Por ello se retira antes el contenedor original, evitando un conflicto de nombre. Esto es posible porque los datos residen en los volúmenes locales y no en el contenedor. El resultado es el archivo `container-neo4j.service` en el directorio de servicios del usuario.

Paso 3. Habilitar e iniciar el servicio

```
systemctl --user daemon-reload
systemctl --user enable --now container-neo4j.service
systemctl --user status container-neo4j.service
```

`daemon-reload` hace que `systemd` lea la nueva unidad; `enable` la registra para el arranque; y `--now` la inicia de inmediato. Finalmente, `status` permite confirmar que el servicio se encuentra en estado *active (running)*. La prueba definitiva consiste en reiniciar la máquina virtual y comprobar, sin iniciar sesión gráfica, que Neo4j responde en el puerto 7474.

Observación: En versiones recientes de Podman, `podman generate systemd` se considera una funcionalidad en desuso, y la alternativa recomendada son los archivos **Quadlet** (`.container`). Para esta práctica se conservó el primer método por su vigencia y simplicidad; migrar a Quadlet queda como mejora futura.

5. Fase 2: Configuración del control de versiones con Git

Git es un sistema de control de versiones distribuido que registra el historial de cambios de un proyecto, permite trabajar en líneas de desarrollo paralelas (ramas) y facilita la colaboración mediante repositorios remotos.

5.1 Instalación y configuración de la identidad

```
sudo apt update
sudo apt install git -y

git config --global user.name "Raúl Guerrero"
git config --global user.email "correo@ejemplo.com"
git config --global --list
```

Git asocia cada *commit* con un autor, por lo que es necesario declarar un nombre y un correo electrónico. La opción `--global` almacena estos valores en `~/.gitconfig`, aplicándolos a todos los repositorios del usuario. El último comando permite verificar la configuración.

5.2 Clonación del repositorio remoto

```
git clone https://github.com/ORGANIZACION/REPOSITORIO-TALLER-BD.git
cd REPOSITORIO-TALLER-BD
```

`git clone` descarga una copia completa del repositorio, incluyendo todo su historial, y configura automáticamente el remoto `origin` apuntando al servidor de origen.

5.3 Sincronización con la rama principal

```
git checkout main
git pull origin main
```

Antes de iniciar cualquier trabajo, se posicionó el repositorio en la rama `main` y se ejecutó `git pull`, que combina `git fetch` (descarga de novedades) con `git merge` (integración en la rama local). Así se garantiza que el punto de partida incluya los cambios más recientes del equipo y se reducen los conflictos posteriores.

5.4 Creación de una rama de trabajo aislada

```
git checkout -b raul-guerrero
git branch
```

`git checkout -b` crea la rama `raul-guerrero` y cambia a ella en un solo paso. Trabajar en una rama propia aísla los cambios del flujo principal: `main` permanece estable mientras el avance individual se desarrolla y se revisa por separado. `git branch` confirma la rama activa, marcada con un asterisco.

5.5 Registro de avances (commit)

```
git status
git add .
git commit -m "Agrega avances de la práctica de Neo4j"
```

El flujo consiste en revisar el estado del árbol de trabajo (`git status`), añadir los cambios al área de preparación (`git add`) y registrarlos en el historial con un mensaje descriptivo (`git commit`). Cada commit constituye un punto de restauración identificable.

5.6 Publicación de la rama y autenticación con PAT

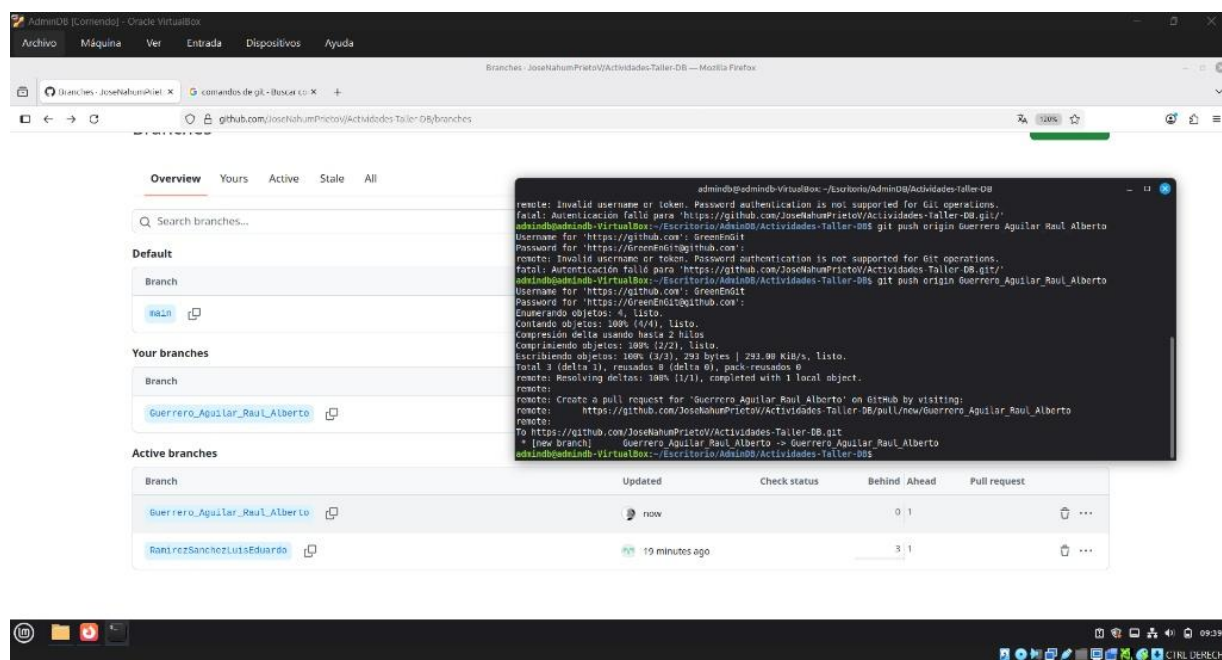
```
git push -u origin raul-guerrero
```

Al intentar subir la rama al servidor remoto, GitHub rechazó la operación. Desde agosto de 2021, GitHub **ya no acepta contraseñas de cuenta** para operaciones Git sobre HTTPS y exige credenciales basadas en token. El mensaje de error indicaba que la autenticación por contraseña había sido eliminada y que se debía usar un Token de Acceso Personal (*Personal Access Token*, PAT).

El problema se resolvió generando un PAT con el siguiente procedimiento:

1. En GitHub, abrir **Settings** → **Developer settings** → **Personal access tokens**.
2. Seleccionar **Generate new token**, asignarle un nombre descriptivo y una fecha de expiración.
3. Conceder el permiso mínimo necesario (por ejemplo, el alcance `repo` en tokens clásicos).
4. Copiar el token inmediatamente, pues GitHub solo lo muestra una vez.
5. Al ejecutar nuevamente `git push`, introducir el **nombre de usuario** de GitHub cuando lo solicite y pegar el **token como contraseña**.

Tras la autenticación, la rama se publicó correctamente y quedó disponible en el repositorio remoto. La opción `-u` estableció el seguimiento (*upstream*) entre la rama local y la remota, por lo que los siguientes envíos y descargas requieren únicamente `git push` y `git pull`.



Buena práctica: El token equivale a una contraseña: no debe compartirse, guardarse en el repositorio ni mostrarse en capturas. Para evitar reescribirlo en cada envío puede usarse un asistente de credenciales, por ejemplo `git config --global credential.helper "cache --timeout=3600"`, que lo conserva en memoria de forma temporal.

6. Fase 3: Implementación de operaciones CRUD en Cypher

6.1 Marco teórico: el ciclo CRUD

CRUD es el acrónimo de las cuatro operaciones básicas de persistencia que cualquier sistema de almacenamiento debe ofrecer: **C**reate (crear), **R**ead (leer), **U**ppdate (actualizar) y **D**elete (eliminar). Constituye el vocabulario mínimo para gestionar el ciclo de vida de un dato: nace, se consulta, se modifica y finalmente se elimina. Aplica a cualquier tipo de base de datos; lo que cambia es el lenguaje con que se expresa. En la siguiente tabla se comparan el modelo relacional y Cypher:

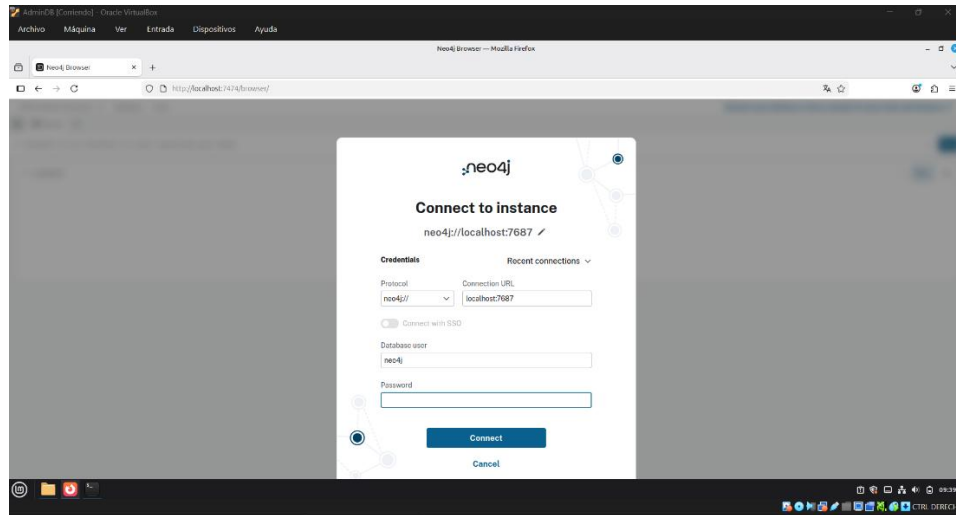
Operación	Significado	SQL (relacional)	Cypher (grafos)
Create	Insertar un nuevo dato	<code>INSERT</code>	<code>CREATE / MERGE</code>
Read	Consultar datos existentes	<code>SELECT</code>	<code>MATCH ... RETURN</code>
Update	Modificar datos existentes	<code>UPDATE</code>	<code>MATCH ... SET</code>
Delete	Eliminar datos	<code>DELETE</code>	<code>MATCH ... DETACH DELETE</code>

Neo4j utiliza el modelo de **grafo de propiedades**. Un nodo puede tener una o varias **etiquetas** (por ejemplo, `Usuario`) que lo clasifican, y un conjunto de **propiedades** en pares clave-valor. Un rasgo importante es que este modelo no impone un esquema rígido: a diferencia de una tabla relacional, agregar un atributo nuevo a un nodo no requiere alterar una estructura previa.

Los patrones en Cypher se escriben entre paréntesis para los nodos. En `(u:Usuario {nombre: 'Raúl'})`, `u` es una variable que referencia al nodo, `Usuario` es su etiqueta y las llaves contienen sus propiedades.

6.2 Acceso a Neo4j Browser

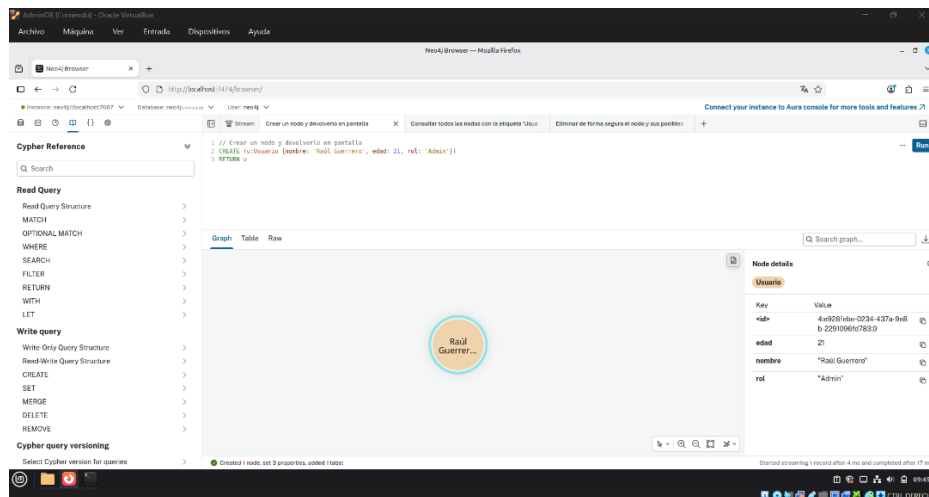
Con el contenedor en ejecución, se accedió a Neo4j Browser desde `http://localhost:7474`. En la pantalla de conexión se seleccionó el protocolo Bolt y se ingresaron las credenciales definidas en la variable `NEO4J_AUTH`. Las sentencias siguientes se escribieron en el editor de consultas del navegador y se ejecutaron sobre un nodo de tipo `Usuario`.



6.3 Create: creación del nodo

```
CREATE (u:Usuario {nombre: 'Raúl Guerrero', edad: 21, rol: 'Admin'})
```

La cláusula `CREATE` inserta un nuevo nodo con la etiqueta `Usuario` y tres propiedades: `nombre`, `edad` y `rol`. Neo4j confirma la operación con un resumen similar a *Added 1 label, created 1 node, set 3 properties*.



6.4 Read: consulta de datos

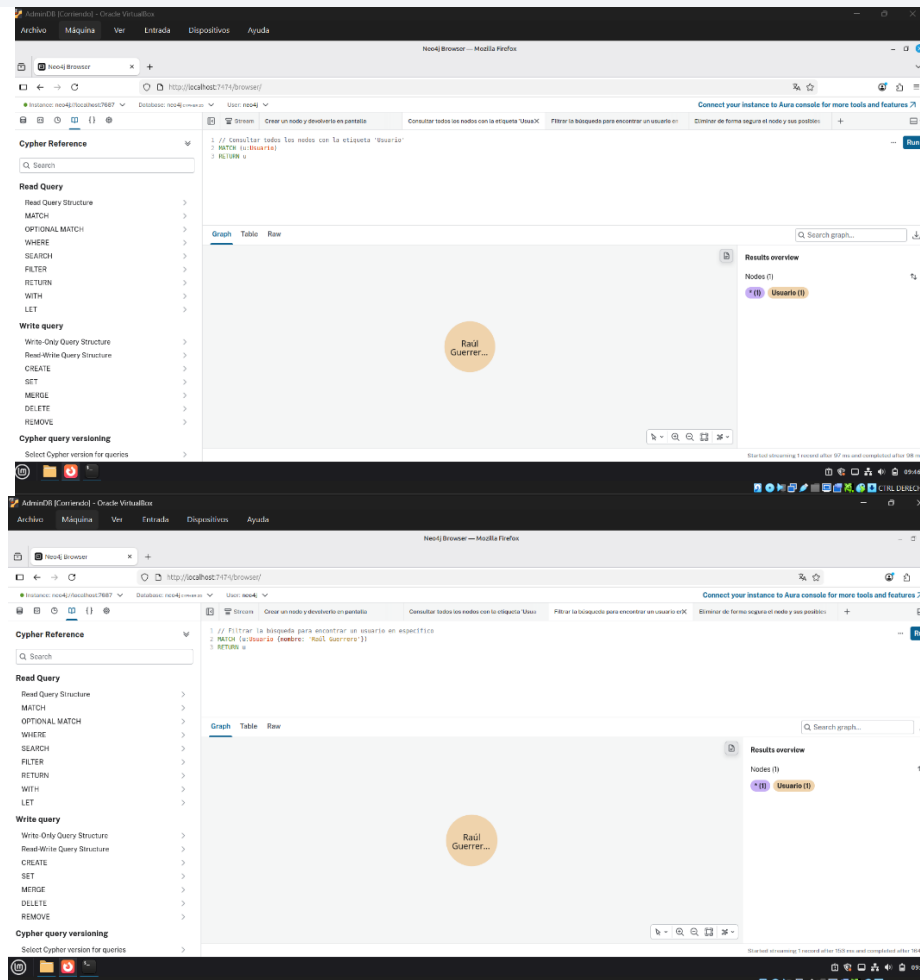
La lectura se realiza con **MATCH**, que describe el patrón a buscar, acompañado de **RETURN**, que define qué se devuelve. Primero se recuperaron todos los nodos de tipo **Usuario**:

```
MATCH (u:Usuario)
RETURN u
```

Después se filtró específicamente el nodo propio, indicando la propiedad dentro del patrón. La condición equivalente con **WHERE** produce el mismo resultado:

```
MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
RETURN u

// Forma equivalente con WHERE
MATCH (u:Usuario)
WHERE u.nombre = 'Raúl Guerrero'
RETURN u
```



Personalización de la interfaz gráfica

Por defecto, Neo4j Browser rotula cada nodo con su identificador interno (`<id>`) o con una propiedad elegida automáticamente, como la edad, lo que dificulta identificar visualmente los nodos. Para mostrar el nombre se ajustó el estilo del grafo:

1. En el panel de resultados, vista de grafo, seleccionar la etiqueta `Usuario` en el panel de resumen.
2. En las opciones de estilo que se despliegan, ubicar el selector **Caption** (rótulo).
3. Elegir la propiedad `nombre`.

Este cambio es únicamente visual: no modifica los datos almacenados en la base, solo la forma en que el navegador los presenta.

6.5 Update: actualización de propiedades

```
MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
SET u.edad = 22,
    u.departamento = 'Sistemas'
RETURN u
```

La actualización requiere primero localizar el nodo con `MATCH` y luego modificarlo con `SET`. Esta sentencia realiza dos acciones distintas con la misma sintaxis: sobrescribe el valor de la propiedad existente `edad` (de 21 a 22) y **añade dinámicamente** la propiedad `departamento` con el valor `Sistemas`, que no existía en el nodo. Esta flexibilidad es consecuencia del modelo sin esquema rígido mencionado anteriormente.

The screenshot shows the Neo4j Browser interface. On the left, the 'Cypher Reference' panel is visible. The main area displays a Cypher query and its results. The query is:

```
1 // Actualizar la edad y agregar el atributo 'departamento'
2 MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
3 SET u.edad = 22, u.departamento = 'Sistemas'
4 RETURN u
```

The results are shown in a graph view. A single node is displayed with the label 'Raúl Guerrero...'. The 'Node details' panel on the right shows the properties of the node:

Key	Value
<code><id></code>	4e928febe-0234-437a-9e8b-2291090fd783.0
<code>departamento</code>	"Sistemas"
<code>edad</code>	22
<code>nombre</code>	"Raúl Guerrero"
<code>rol</code>	"Admin"

6.6 Delete: eliminación segura del nodo

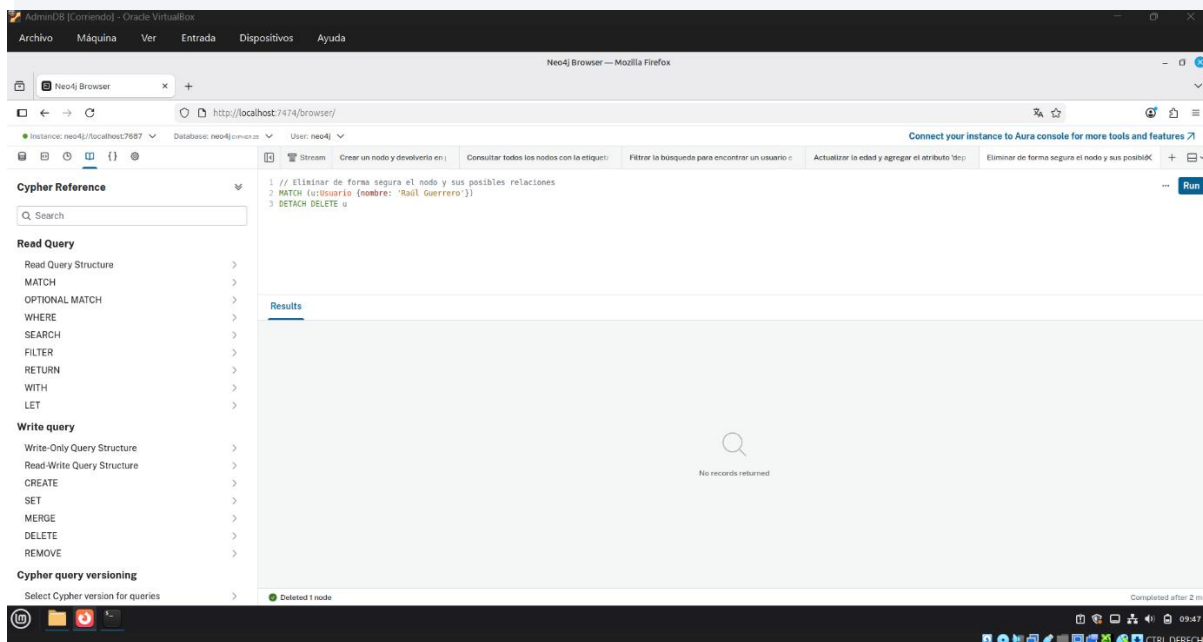
```
MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
DETACH DELETE u
```

En una base de datos de grafos, un nodo puede estar conectado a otros mediante relaciones, y una relación no puede existir sin sus dos extremos. Por esta razón, Neo4j protege la integridad del grafo: la cláusula **DELETE** por sí sola **rechaza** la eliminación de un nodo que aún tenga relaciones asociadas.

DETACH DELETE resuelve esta situación de manera segura: primero **desvincula** el nodo eliminando todas sus relaciones y, a continuación, elimina el nodo. Así no quedan relaciones huérfanas ni es necesario borrarlas manualmente una por una. Para ilustrar el contraste:

```
// Falla si el nodo tiene relaciones asociadas
MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
DELETE u

// Elimina el nodo y todas sus relaciones
MATCH (u:Usuario {nombre: 'Raúl Guerrero'})
DETACH DELETE u
```



7. Resumen de resultados

Fase	Actividad	Resultado
1	Despliegue de Neo4j con Podman (rootless) y volúmenes persistentes	Instancia operativa en los puertos 7474 y 7687 con datos persistentes.
1	Automatización con <code>loginctl enable-linger</code> y <code>podman generate systemd</code>	Neo4j inicia con el sistema, sin requerir sesión de usuario.
2	Instalación y configuración de Git; clonación y sincronización de <code>main</code>	Repositorio local actualizado y con identidad configurada.
2	Rama <code>raul-guerrero</code> , commit y push con PAT	Avances publicados en el remoto sin afectar <code>main</code> .
3	Operaciones <code>CREATE</code> , <code>MATCH</code> , <code>SET</code> y <code>DETACH DELETE</code> sobre <code>Usuario</code>	Ciclo CRUD completo verificado en Neo4j Browser.

8. Conclusiones

La práctica permitió integrar tres piezas que, combinadas, conforman un flujo de trabajo profesional: un motor de base de datos encapsulado en un contenedor, un arranque automatizado y un historial de cambios versionado.

- **Beneficios del uso de contenedores automatizados.** Desplegar Neo4j con Podman evita instalar y configurar el motor directamente en el sistema, lo que mantiene limpio el anfitrión y hace el entorno reproducible y desechable: basta con recrear el contenedor para volver a un estado conocido.
- **Beneficios de integrarlos con el control de versiones.** Git aporta trazabilidad, colaboración y seguridad en el trabajo: las ramas aisladas protegen la línea principal, cada commit queda registrado con su autor y su propósito, y el repositorio remoto funciona como respaldo y punto de intercambio
- **Dominio del ciclo CRUD en grafos.** La ejecución de `CREATE`, `MATCH`, `SET` y `DETACH DELETE` mostró la expresividad de Cypher y dos características propias del modelo de grafos: la flexibilidad para añadir propiedades sin un esquema rígido y la protección de la integridad referencial al eliminar nodos con relaciones.
- **Lecciones aprendidas y mejoras futuras.** Los problemas resueltos durante la práctica (saltos de línea al pegar comandos, directorios inexistentes y autenticación con token) son obstáculos habituales y su solución dejó buenas prácticas claras: preparar el entorno antes de desplegar, verificar cada paso y tratar las credenciales como información sensible.